

TECHNICAL REPORT

IoT / EMBEDDED SYSTEMS

Project 5: FreeRTOS Task Separation, Sensor Task and Output Task

Daily Technical Report | Day 26 + Day 27

Platform	ESP32 · ESP-IDF · PlatformIO
Subsystem	Two independent FreeRTOS tasks with shared global data, potentiometer to PWM and IR sensor to digital LED
Prepare by	Malak Alsharqawi
Prepared For	IoT Training Program

Contents

1. Daily Overview	4
1.1 Summary	4
1.2 Tasks Completed	4
2. FreeRTOS Fundamentals	5
2.1 What FreeRTOS Is	5
2.2 General Purpose OS Compared With a Real-Time OS	5
2.3 FreeRTOS Compared With an Ordinary Operating System	6
2.4 How FreeRTOS Relates to ESP-IDF	6
3. The Scheduler and the Five Task States	7
3.1 What the Scheduler Does	7
3.2 Concurrency and Parallelism	7
3.3 The Five Task States	7
3.4 Why Blocking Is Better Than Busy Waiting	9
3.5 Context Switching and Preemption	9
3.6 The RTOS Tick and pdMS_TO_TICKS()	9
3.7 The Idle Task	10
4. Task Priority	10
4.1 What Priority Actually Decides	10
4.2 State and Priority Work Together	10
4.3 Task Starvation	11
4.4 Why the Output Task Was Given the Higher Priority	11
5. The app_main() Lifecycle	11
6. Logging with ESP_LOG	12
6.1 Why Not printf	12
6.2 The Three Severity Levels	13
6.3 One TAG per Task	13
6.4 Baud Rate: A Separate Concept From Logging	13
7. Hardware and Circuit	14
7.1 Wiring Diagram	14
7.2 Connections Shown in the Diagram	15
7.3 Verification Note on the Diagram	15
8. Firmware Implementation	15
8.1 Architecture	15
8.2 Full Listing	16
8.3 Walkthrough	18
8.4 Timing and Why the System Feels Faster	19
9. Verification and Serial Monitor Evidence	19
9.1 What Counts as Proof of Independent Scheduling	19
9.2 Optional: vTaskGetRunTimeStats()	20
9.3 Suggested Improvement to the Logging	20
10. Report Questions	20
10.1 The Five Task States, and Where Our Tasks Spend Their Time	20
10.2 What Priority Decides When Two Tasks Are Both Ready	21
10.3 How the Output Task Can Read Stale or Inconsistent Data	3

10.4 Why the Dual Core Architecture Matters Here.....	3
11. Photographic Appendix.....	3
12. Definition of Done	3
12.1 Carried Forward to the Next Task	3
13. References	3

1. Daily Overview

1.1 Summary

This session converted the Task 15 single loop firmware into a two task FreeRTOS application. In the previous version, one execution loop was responsible for everything: read the potentiometer, read the IR sensor, update the PWM duty cycle, and update the digital LED. Every one of those actions was serialised behind the same delay, so the responsiveness of the whole system was dictated by the slowest thing in the loop.

The program now runs as two independent tasks created with `xTaskCreate()`. The Sensor Task owns input acquisition only: it reads the potentiometer through the ADC oneshot driver and reads the IR sensor through a GPIO input. The Output Task owns actuation only: it applies the potentiometer value to the LEDC PWM channel that drives LED brightness, and it drives the digital LED from the IR sensor state. Neither task calls into the other. They communicate through two plain global integers, which is deliberately the naive and unsafe method the assignment asks for at this stage.

Alongside the restructuring, every remaining `printf` call was removed. All console output now goes through the ESP-IDF logging macros `ESP_LOGI`, `ESP_LOGW` and `ESP_LOGE`, with a separate TAG per task so that any line in the Serial Monitor can be attributed to its source. The interleaving of the two TAGs in the log, together with the timestamp that ESP-IDF prepends to every line, is what provides the evidence that the two tasks are genuinely being scheduled independently rather than running one after the other.

The second half of the session was theory: what the scheduler actually is, the five task states and the transitions between them, what a priority number really decides, why blocking is better than busy waiting, how `app_main` fits into the FreeRTOS lifecycle, and why the ESP32 being a dual core part changes the shared data question in a way that a single core FreeRTOS port would not.

1.2 Tasks Completed

- Split the Task 15 single loop program into a Sensor Task and an Output Task, each created with `xTaskCreate()` and each given its own stack size and priority.
- Assigned Output Task the higher priority (2) and Sensor Task the lower priority (1), and confirmed by reasoning and by log evidence why the lower priority task still gets CPU time.
- Gave each task its own infinite loop with a `vTaskDelay()` at the end, so that no task busy waits and every task spends most of its cycle in the Blocked state.
- Shared the two sensor readings between the tasks using plain global integers, with no Queue, Mutex or Semaphore, and documented the failure modes this creates rather than fixing them.
- Removed all `printf` calls and replaced them with `ESP_LOGI` for normal values, `ESP_LOGE` for a failed ADC read, and reserved `ESP_LOGW` for out of range readings.
- Defined a distinct TAG for each task (`SENSOR_TASK` and `OUTPUT_TASK`) so the Serial Monitor identifies the source of every line.
- Captured Serial Monitor output showing `SENSOR_TASK` and `OUTPUT_TASK` lines interleaving with advancing timestamps, as proof of independent scheduling.

- Reduced the task periods from the earlier 1000 ms / 500 ms down to 100 ms for the Sensor Task and 50 ms for the Output Task, and confirmed that the improvement in responsiveness came from the delay change and not from the priority change.
- Studied the FreeRTOS scheduler, the five task states, context switching, preemption, the RTOS tick, the Idle Task, and the relationship between ESP-IDF and FreeRTOS.
- Answered the four report questions on task states, priority semantics, unsynchronised shared data, and dual core implications.

2. FreeRTOS Fundamentals

2.1 What FreeRTOS Is

FreeRTOS stands for Free Real-Time Operating System. It is a lightweight operating system kernel intended for microcontrollers such as the ESP32. It deliberately does not provide the things a desktop operating system provides: there are no windows, no user accounts, no file explorer, no graphical interface and no general application layer. Its job is narrow. It lets one firmware image be divided into several independent units of work, and it decides which of those units gets the CPU at any given moment.

Before this session, the firmware looked like the pattern below, where a single loop carried every responsibility:

```
while (1)
{
    Read sensors
    Control motor
    Update LEDs
    Check Wi-Fi
    Send MQTT
}
```

With FreeRTOS the same responsibilities become separate tasks, and a component called the scheduler decides which one runs:

```
Task 1 -> Read sensors
Task 2 -> Control outputs
Task 3 -> Handle Wi-Fi
Task 4 -> Safety monitoring
```

2.2 General Purpose OS Compared With a Real-Time OS

Windows, Linux and macOS are general purpose operating systems. They manage CPU time, memory, storage, files, USB devices, networking, processes, permissions, users, displays and audio. FreeRTOS performs a conceptually similar management role, but at a far smaller scale and with a different objective.

The difference that matters is the meaning of the word real-time. Real-time does not mean fast. It means that the system responds within a predictable and bounded amount of time.

A system can be extremely fast and still not be real-time, if its response time is unpredictable.

Consider an industrial machine where a safety sensor detects a dangerous condition and the motor must stop within 5 ms. The requirement is not that the system should respond quickly. The requirement is that the system must respond before the deadline, every time. On a desktop operating system a mouse click might be

handled in 10 ms, or 70 ms, or after a short stall while the system is busy. For browsing that is perfectly acceptable. For a motor controller, an industrial machine, a robot or a safety interlock it is not.

2.3 FreeRTOS Compared With an Ordinary Operating System

Feature	FreeRTOS	Windows / Linux
Main purpose	Embedded and real-time systems	General purpose computing
Typical hardware	Microcontrollers	PCs and servers
Size	Very small	Very large
RAM requirement	KB to low MB range	Usually GB range
Main execution unit	Tasks	Processes and threads
Timing focus	Predictable response	General responsiveness and throughput
Task isolation	Usually limited, shared address space	Strong process isolation
Priorities	Central to scheduling	Present, but less central for normal applications
Graphical interface	Normally none	Usually available
File system	Optional and simple	Full featured
Resource usage	Very low	Much higher

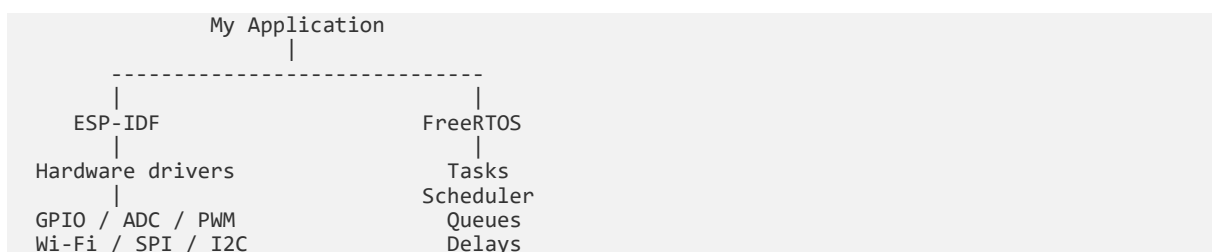
Table 1. FreeRTOS positioned against a general purpose operating system.

Memory isolation is the important asymmetry

On a desktop operating system, if Chrome crashes, Windows keeps running, because one application cannot normally overwrite another application's memory. In a typical FreeRTOS microcontroller application all tasks are part of the same firmware image and share the same address space. If one task writes out of bounds it can corrupt another task's data or crash the whole program. This is the reason embedded programming demands careful memory handling, and it is also the reason the shared global variables used in this project are a real hazard rather than a theoretical one.

2.4 How FreeRTOS Relates to ESP-IDF

Using FreeRTOS does not mean replacing ESP-IDF. They work together and they solve different problems. ESP-IDF supplies the hardware drivers and system APIs, for example `gpio_get_level()`, `adc_oneshot_read()` and `ledc_set_duty()`, which reach GPIO, ADC, PWM, Wi-Fi, SPI, I2C and UART. FreeRTOS supplies the multitasking primitives, for example `xTaskCreate()` and `vTaskDelay()`.



ESP-IDF already includes and runs FreeRTOS. We are not installing it in this task, we are starting to use it directly.

A common misunderstanding is that the ESP32 was not running an operating system before FreeRTOS tasks were created. That is not correct under ESP-IDF. FreeRTOS starts as part of system initialisation, and ESP-IDF then calls `app_main()` from inside a FreeRTOS task, referred to as the main task. Even the earlier single loop firmware was already executing inside a FreeRTOS task without the code being aware of it.

3. The Scheduler and the Five Task States

3.1 What the Scheduler Does

When a program contains several tasks, the CPU cannot simply execute all of them at once. Something has to answer the question of which task should run right now. That component is the scheduler, and it is part of the FreeRTOS kernel. Its responsibilities are to track every task in the system, know which tasks are ready to run, know which tasks are waiting, compare task priorities, select the appropriate task, and switch the CPU from one task to another.

The reason a scheduler is needed becomes obvious as soon as tasks contain infinite loops. If three task functions were simply called one after another in plain C, and the first one contained a `while (1)` loop, execution would never reach the second or the third. FreeRTOS solves this by owning the tasks rather than letting one function own the CPU forever.

```

app_main()
|
v
Sensor Task
|
while(1)
|
+----- never ends

Motor Task  -> NEVER REACHED
MQTT Task   -> NEVER REACHED

```

3.2 Concurrency and Parallelism

On a single CPU core only one task can actually execute instructions at any given instant. The scheduler switches between tasks quickly enough that they appear to run simultaneously, and this appearance is called concurrency. If the processor has more than one core, some tasks may genuinely execute at the same instant, and that is parallelism. The ESP32 has two cores, so both effects are present in this project. Section 10.4 returns to what that means for the shared variables.

3.3 The Five Task States

The scheduler does not treat a task as simply running or not running. Every task carries a state, and the state determines whether the task is even a candidate for the CPU.

```

+-----+
|  RUNNING  |
+-----+
|
scheduler switches

```

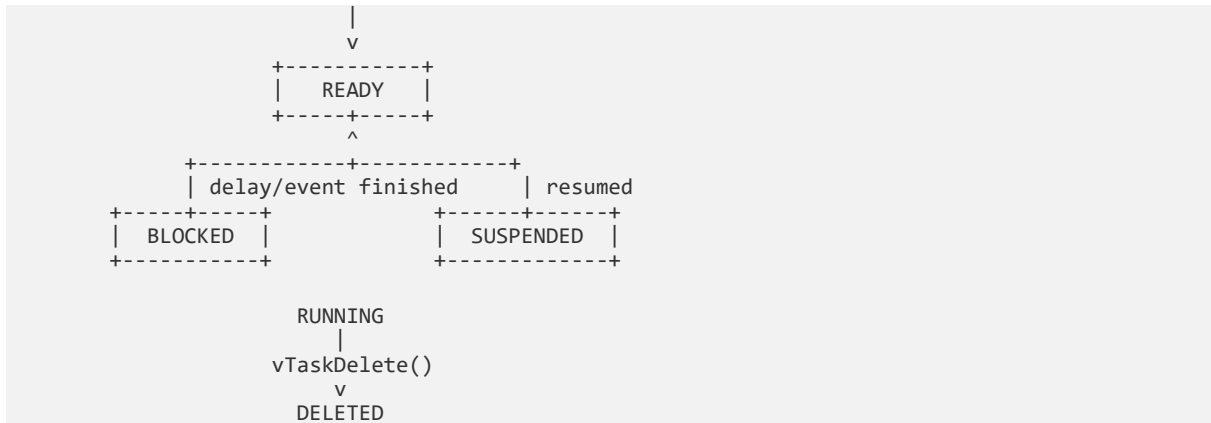


Figure 1. Task state model. The four schedulable states plus the Deleted lifecycle end point.

State	Meaning	Wants CPU?	Typical cause in this project
Running	The CPU is currently executing this task's instructions	Yes	adc_oneshot_read() executing inside sensor_task
Ready	Able to run, but another task currently owns the CPU	Yes	Sensor Task waiting while the higher priority Output Task runs
Blocked	Waiting for a time delay or an event, so it does not want the CPU	No	vTaskDelay(pdMS_TO_TICKS(100)) in the Sensor Task
Suspended	Manually removed from scheduling until explicitly resumed	No	vTaskSuspend(), not used in this project
Deleted	The task has been removed and no longer participates in scheduling	No	vTaskDelete(), not used here since both tasks run forever

Table 2. The five task states.

Running

Only one task can be Running per core at a time. On the dual core ESP32 there can be one Running task per core, so two tasks can be Running simultaneously.

Ready

A Ready task is not sleeping and is not delayed. It is willing and able to execute, it simply does not currently own the CPU. The mental phrasing that keeps this straight is: I can run now, but something else has the CPU.

Blocked

A Blocked task is waiting for something: a time delay, data from a queue, a semaphore, a notification, or an event. In this project the reason is always vTaskDelay(). When the Sensor Task executes vTaskDelay(pdMS_TO_TICKS(100)) it is telling the kernel not to schedule it for the next 100 ms. The phrasing here is: I am waiting, and I do not need CPU time yet.

Suspended

A suspended task has been manually stopped with `vTaskSuspend()` and, unlike a delay, will not become Ready again by itself. Another task has to call `vTaskResume()`. Blocked tasks are waiting for an event or a timeout, suspended tasks are intentionally disabled.

Deleted

`vTaskDelete()` removes a task from scheduling entirely. `vTaskDelete(NULL)` means the currently running task deletes itself. Neither task in this project is deleted, because both are intended to run for the lifetime of the device. A FreeRTOS task must never simply fall off the end of its function. If a task is genuinely meant to finish, it has to call `vTaskDelete(NULL)` explicitly.

3.4 Why Blocking Is Better Than Busy Waiting

If a task is written as a bare loop with no delay, it continuously requests CPU time even when it has no useful work to perform. That is busy waiting, and it wastes cycles that another task could have used.

```
Busy waiting:
Sensor Task
#####

With vTaskDelay():
Sensor Task
RUN      BLOCKED      RUN      BLOCKED
####    - - - - -    ####    - - - - -
```

During the Blocked period the kernel is free to run other work. This is why the assignment requires a `vTaskDelay()` inside every task loop, and why there is no while (1) busy wait anywhere in the project.

3.5 Context Switching and Preemption

When the CPU moves from one task to another, FreeRTOS performs a context switch. It saves the outgoing task's CPU registers, stack pointer, execution position and state, then restores the incoming task's saved context. The incoming task does not restart from the beginning, it continues from where it was interrupted.

Two events commonly trigger the scheduler to reconsider. The first is a task becoming Blocked, for example by calling `vTaskDelay()`, which frees the CPU immediately. The second is a higher priority task becoming Ready while a lower priority task is running. FreeRTOS uses preemptive scheduling by default, which means the running lower priority task is interrupted rather than being allowed to finish voluntarily. That interruption is called preemption, and it is one of the main reasons FreeRTOS is usable for time sensitive systems.

3.6 The RTOS Tick and `pdMS_TO_TICKS()`

FreeRTOS maintains a periodic timing interrupt known as the tick, which acts as the kernel's clock and drives delays and timeouts. The tick period depends on the FreeRTOS configuration, so a delay expressed in raw ticks is not portable.

```
vTaskDelay(100);           // 100 ticks, NOT necessarily 100 ms
vTaskDelay(pdMS_TO_TICKS(100)); // approximately 100 milliseconds
```

`pdMS_TO_TICKS()` converts milliseconds into the correct number of ticks for the configured tick rate. Using it states the intent directly and survives a change to the tick frequency, which is why every delay in this project is written that way.

3.7 The Idle Task

If every application task is Blocked at the same moment, FreeRTOS runs a special task it creates itself, the Idle Task. Seeing the Idle Task run is not a sign that the program is broken. In a well written application it is normal, and it usually means the other tasks are correctly waiting instead of wasting CPU time.

4. Task Priority

4.1 What Priority Actually Decides

Every FreeRTOS task is assigned a priority when it is created. In this project the priority is the fifth argument to `xTaskCreate()`: 1 for the Sensor Task and 2 for the Output Task. FreeRTOS priorities start at 0 and increase upward, so a larger number means a higher priority.

A higher priority gives a task preference when several tasks are Ready at the same time. That is the whole of it.

Priority does not make a task run faster, and it does not make a task run more often. It is not a speed multiplier and it is not a frequency setting. The only question priority answers is which of the currently Ready tasks the scheduler should hand the CPU to.

```
Priority      -> Who gets preference when tasks are Ready?
Delay / Period -> How often does the task wake up?

Priority != Speed
Priority != Frequency
Priority != Delay
```

4.2 State and Priority Work Together

The scheduler never looks at priority alone. It first looks at state, then compares priorities among the tasks that are actually Ready.

Task	Priority	State	Result
Output Task	2	Blocked	Cannot run, it is waiting on its delay
Sensor Task	1	Ready	Runs, despite being the lower priority task
Output Task	2	Ready	Runs, it is the highest priority Ready task
Sensor Task	1	Ready	Waits in Ready until the CPU is free

Table 3. A lower priority task runs whenever every higher priority task is unavailable.

This is exactly what happens in the running system. The Output Task has the higher priority, but it calls `vTaskDelay(pdMS_TO_TICKS(50))` on every pass, so it becomes Blocked regularly and releases the CPU. During those windows the Sensor Task is the highest priority Ready task and it runs.

4.3 Task Starvation

If the higher priority Output Task were written without any delay or other blocking call, it would remain permanently Ready and would keep winning the scheduling decision. The Sensor Task would be Ready over and over and would rarely or never receive CPU time. That condition is called task starvation.

```
Output Task (no delay):
#####

Sensor Task:
.....
```

A well designed FreeRTOS task blocks or delays whenever it has no useful work to do. This is the second reason, alongside CPU efficiency, that both tasks in this project end their loop with `vTaskDelay()`.

4.4 Why the Output Task Was Given the Higher Priority

The assignment specifies that the Output Task should run at the higher priority, mirroring the reference guide's example. The design reasoning is that once an output update is ready to be applied, it should be applied with as little scheduling delay as possible, since it is the stage that is physically visible to the user. A stale ADC sample that arrives a few milliseconds late is harmless. An output update that is queued behind other work is what makes a system feel sluggish.

It should be said plainly that this is primarily an exercise in demonstrating how scheduling priorities behave. It does not follow that an output task should always outrank a sensor task in every embedded design. In a system where the sensor is a safety input, the ordering would usually be the other way around.

5. The `app_main()` Lifecycle

A question that comes up immediately when tasks are created inside `app_main()` is whether `app_main()` has to keep running for those tasks to survive. It does not.

`app_main()` creates tasks and is then allowed to finish. FreeRTOS stays alive and continues scheduling every task that still exists.

The startup order on an ESP-IDF project is as follows. The ESP32 powers on, ESP-IDF performs system initialisation, the FreeRTOS scheduler starts, ESP-IDF creates the main task, and that main task calls `app_main()`. By the time application code executes, FreeRTOS has already been running for some time.

```
ESP32 boots
|
ESP-IDF initialisation
|
FreeRTOS scheduler is running
|
ESP-IDF main task
|
app_main()
|
xTaskCreate(sensor_task) ----> Sensor Task created
xTaskCreate(output_task) ----> Output Task created
|
app_main() returns, main task finishes
|
FreeRTOS continues scheduling Sensor Task and Output Task
```

This differs from the pthread model used in earlier C projects. In a Linux program, threads belong to a process. When `main()` returns the process terminates and every thread disappears with it, which is why `pthread_join()` is normally required. Under ESP-IDF, tasks belong to the FreeRTOS scheduler rather than to `app_main()`. Returning from `app_main()` ends only the main task. The scheduler itself does not stop, and the remaining tasks continue.

Aspect	Philosophers / pthread	ESP32 / FreeRTOS
Container	A Linux process contains threads	FreeRTOS manages tasks
Creation	<code>pthread_create()</code> creates a thread	<code>xTaskCreate()</code> creates a task
Waiting	Main thread usually waits with <code>pthread_join()</code>	<code>app_main()</code> does not need to wait
Returning from entry	Returning from <code>main()</code> terminates the process	Returning from <code>app_main()</code> does not stop FreeRTOS
Effect on siblings	Other threads disappear when the process exits	Other tasks continue running
Lifetime owner	<code>main()</code> usually controls process lifetime	The scheduler controls task execution

Table 4. Why pthread habits do not carry over directly to FreeRTOS.

A practical consequence is that adding an empty while (1) loop at the end of `app_main()` purely to keep the program alive is wrong on both counts: it is unnecessary, and it is a busy wait that consumes CPU time for no benefit. The infinite loop belongs inside each task that has to keep working, not in `app_main()`.

6. Logging with ESP_LOG

6.1 Why Not printf

ESP-IDF provides a logging system through `esp_log.h` that is better suited to embedded work than raw `printf`. A `printf` call produces only the message. A log macro produces a severity level, a timestamp, a source TAG and the message.

```
printf("ADC raw = %d\n", adc_raw);
-> ADC raw = 1850

ESP_LOGI(TAG, "ADC raw = %d", adc_raw);
-> I (1023) SENSOR_TASK: ADC raw = 1850
```

Each part of that line carries information. The leading I is the severity level, (1023) is the timestamp in milliseconds since boot, `SENSOR_TASK` is the TAG identifying which part of the program emitted the line, and the remainder is the message. In a FreeRTOS project where several tasks print concurrently, this is the difference between a readable log and a stream of ambiguous text. The project contains zero raw `printf` calls.

6.2 The Three Severity Levels

Macro	Meaning	When it is used in this project
ESP_LOGI()	Information	Normal values and normal behaviour: potentiometer raw value, IR sensor state, PWM duty being applied, task running confirmation
ESP_LOGW()	Warning	Unusual but recoverable: an ADC reading sitting at or near the top of its range, a value outside the expected band, a retried operation
ESP_LOGE()	Error	An operation that was expected to succeed has failed: adc_oneshot_read() returning something other than ESP_OK, a driver configuration failure, a failed task creation

Table 5. Severity selection rules applied throughout the firmware.

The distinction is one of severity, not of formatting. The system working normally is Information. Something suspicious but survivable is a Warning. Something that should have worked and did not is an Error. Overusing Error for ordinary conditions destroys the value of the level, which is why the ADC path in this firmware logs Information on success and Error only on a genuine driver failure.

6.3 One TAG per Task

Each task defines its own TAG, so every log line is attributable at a glance.

```
static const char *SENSOR_TAG = "SENSOR_TASK";
static const char *OUTPUT_TAG = "OUTPUT_TASK";

ESP_LOGI(SENSOR_TAG, "Potentiometer raw value: %d", adc_raw);
ESP_LOGI(SENSOR_TAG, "IR sensor state: %d", ir_state);
ESP_LOGI(OUTPUT_TAG, "PWM duty = %d", pwm_duty);
```

6.4 Baud Rate: A Separate Concept From Logging

The baud rate is the speed of the serial connection between the ESP32 and the computer, and both ends must agree on it. If the ESP-IDF console is configured for 115200 and the PlatformIO monitor is set to 9600, the computer decodes the incoming bits with the wrong timing and the Serial Monitor fills with unreadable characters. The firmware is running correctly in that situation; only the decoding is wrong.

It is worth separating two things that are easy to conflate. ESP_LOGI() decides what is printed. The ESP-IDF console configuration decides how fast it is transmitted. The macro does not itself select 115200.

```
ESP_LOGI()
  |
  v
Formatted log message
  |
  v
ESP-IDF logging system -> ESP-IDF console
  |
  | configured baud rate
  v
UART / USB serial -> PlatformIO Serial Monitor
```

115200 is the common default for ESP-IDF development because it balances speed, reliability and compatibility, and it comfortably carries a project that emits many log lines per second. The PlatformIO side is matched in `platformio.ini`:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
monitor_speed = 115200
```

In a pure ESP-IDF project there is no `Serial.begin(115200)` call. That is an Arduino idiom. ESP-IDF manages its own console, and the application only needs to emit log macros.

7. Hardware and Circuit

7.1 Wiring Diagram

The circuit below is the build used for this task, drawn in Circuit Designer. The ESP32 development board supplies 3V3 and GND to the breadboard power rails. A rotary potentiometer forms the analog input, and two LEDs form the output stage: one driven by the PWM channel for brightness control, and one driven as a plain digital output.

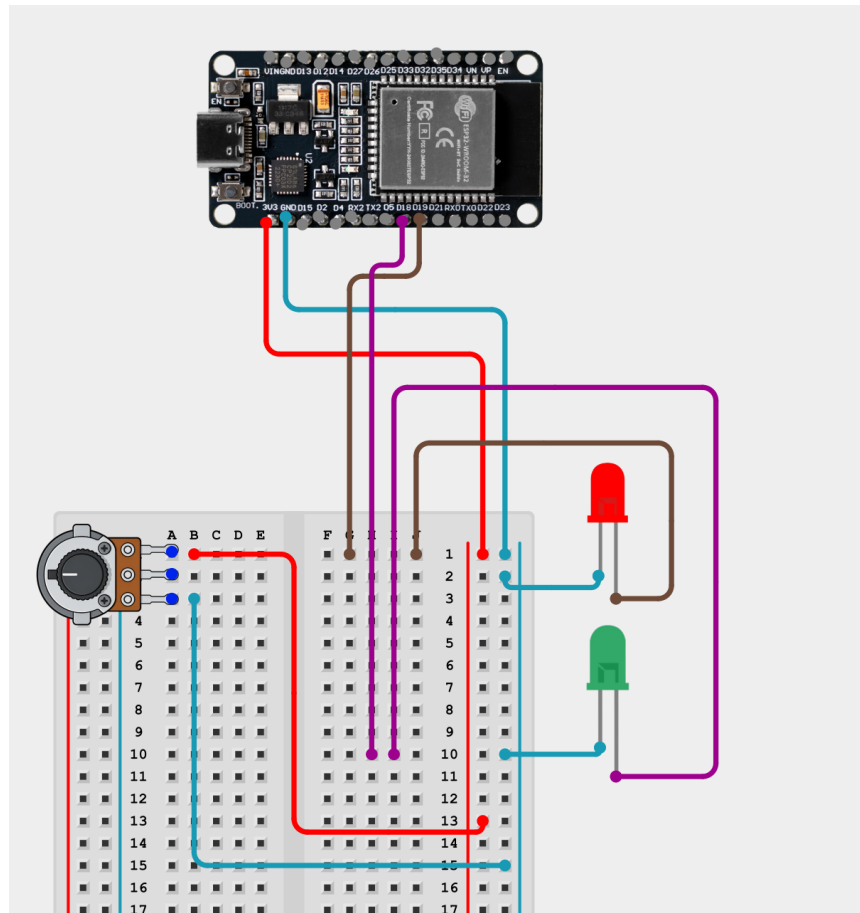


Figure 2. Wiring diagram: ESP32 development board, potentiometer, and two indicator LEDs on a breadboard.

7.2 Connections Shown in the Diagram

Wire	From (ESP32)	To
Red	3V3	Breadboard positive rail; the rail then feeds the top terminal of the potentiometer
Cyan	GND	Breadboard ground rail; the rail feeds the bottom terminal of the potentiometer and the cathode of each LED
Purple	GPIO 18 (D18)	Breadboard node in the F to J half, row 10, which continues to the anode of the green LED
Brown	GPIO 19 (D19)	Breadboard node in the F to J half, row 1, which continues to the anode of the red LED

Table 6. Wire by wire reading of the diagram in Figure 2.

7.3 Verification Note on the Diagram

Two items in the firmware are not represented in the diagram as drawn, and they should be added before this figure is submitted as the definitive schematic. Flagging them here is deliberate, so that the report and the hardware do not disagree.

- The potentiometer wiper, which is the centre terminal, has no wire attached in the diagram. Only the two end terminals are connected, to 3V3 and to GND. As drawn, the potentiometer is a voltage divider with no output tap, so no analog value would reach the ESP32. The wiper needs to be wired to the ADC1 channel referenced by POT_CHANNEL in header.h. ADC1 should be used rather than ADC2, since ADC2 is shared with the Wi-Fi subsystem.
- The IR obstacle sensor is not present in the diagram at all. It needs a three wire connection: VCC to the 3V3 rail, GND to the ground rail, and its digital output to the GPIO referenced by IR_GPIO in header.h.
- Current limiting resistors are not shown in series with either LED. If they are absent on the physical build rather than merely omitted from the drawing, they should be added, typically in the 220 to 330 ohm range for a standard indicator LED on a 3.3 V rail.

Which of the two GPIOs carries the PWM signal and which carries the digital output is fixed by the PWM_LED_GPIO and BLINK_GPIO definitions in header.h, which is not reproduced in the Notion page. The diagram shows GPIO 18 going to the green LED and GPIO 19 to the red LED; the mapping to the two roles should be confirmed against the header before this table is treated as final.

8. Firmware Implementation

8.1 Architecture



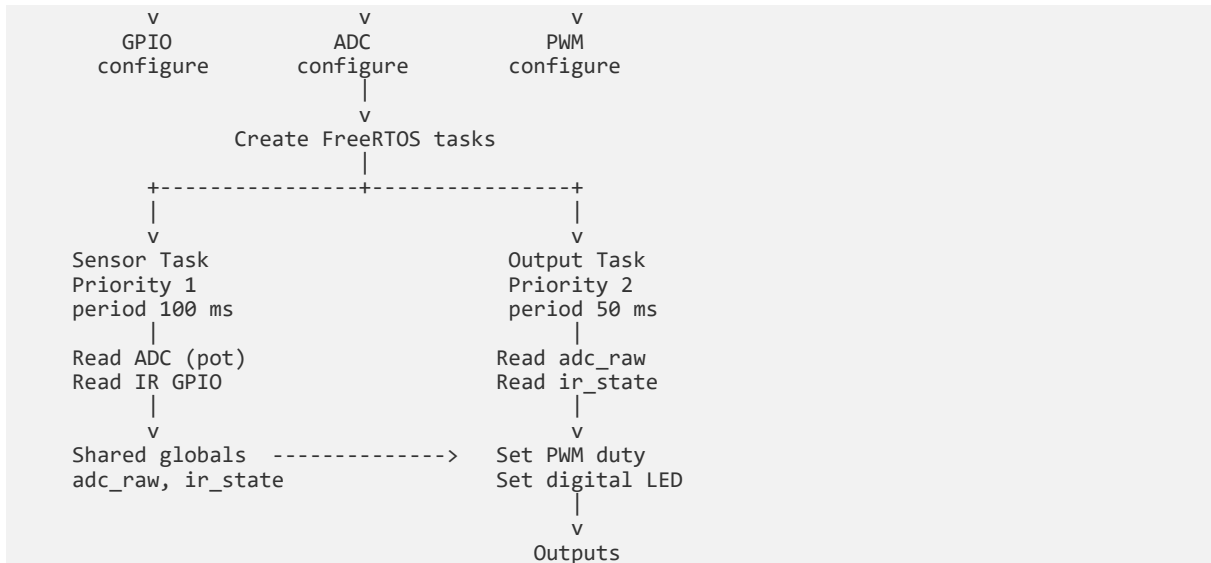


Figure 3. Current program architecture. Data flows one way, from the Sensor Task through global memory to the Output Task.

8.2 Full Listing

The complete firmware as it stands at the end of Day 27 is reproduced below.

```

#include "header.h"

static const char *OUTPUT_TAG = "OUTPUT_TASK";
static const char *SENSOR_TAG = "SENSOR_TASK";

int adc_raw;
int ir_state;

adc_oneshot_unit_handle_t adc1_handle;

void pwm_setup()
{
    ledc_timer_config_t pwm_timer = {
        .speed_mode    = LEDC_LOW_SPEED_MODE,
        .timer_num     = LEDC_TIMER_0,
        .duty_resolution = LEDC_TIMER_12_BIT,
        .freq_hz       = 5000,
        .clk_cfg       = LEDC_AUTO_CLK
    };

    ledc_timer_config(&pwm_timer);

    ledc_channel_config_t pwm_channel = {
        .gpio_num    = PWM_LED_GPIO,
        .speed_mode  = LEDC_LOW_SPEED_MODE,
        .channel     = LEDC_CHANNEL_0,
        .intr_type   = LEDC_INTR_DISABLE,
        .timer_sel   = LEDC_TIMER_0,
        .duty        = 0,
        .hpoint      = 0
    };

    ledc_channel_config(&pwm_channel);
}

void setup()
{

```

```
    gpio_reset_pin(IR_GPIO);
    gpio_reset_pin(BLINK_GPIO);

    gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);
    gpio_set_direction(IR_GPIO, GPIO_MODE_INPUT);

    adc_oneshot_unit_init_cfg_t init_config = {
        .unit_id = ADC_UNIT_1
    };

    adc_oneshot_new_unit(&init_config, &adc1_handle);

    adc_oneshot_chan_cfg_t config = {
        .bitwidth = ADC_BITWIDTH_DEFAULT,
        .atten    = ADC_ATTEN_DB_12
    };

    adc_oneshot_config_channel(adc1_handle, POT_CHANNEL, &config);

    pwm_setup();
}

void sensor_task(void *pvParameters)
{
    esp_err_t ret;

    while (1)
    {
        ret = adc_oneshot_read(adc1_handle, POT_CHANNEL, &adc_raw);

        if (ret == ESP_OK)
            ESP_LOGI(SENSOR_TAG, "Potentiometer raw value: %d", adc_raw);
        else
            ESP_LOGE(SENSOR_TAG, "ADC read failed");

        ir_state = gpio_get_level(IR_GPIO);

        ESP_LOGI(SENSOR_TAG, "IR sensor state: %d", ir_state);

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void output_task(void *pvParameters)
{
    while (1)
    {
        ledc_set_duty(LEDCLOW_SPEED_MODE, LEDC_CHANNEL_0, adc_raw);
        ledc_update_duty(LEDCLOW_SPEED_MODE, LEDC_CHANNEL_0);

        gpio_set_level(BLINK_GPIO, !ir_state);

        ESP_LOGI(OUTPUT_TAG, "Output task is running");

        vTaskDelay(pdMS_TO_TICKS(50));
    }
}

void app_main()
{
    setup();

    xTaskCreate(sensor_task, "Sensor Task", 2048, NULL, 1, NULL);
    xTaskCreate(output_task, "Output Task", 2048, NULL, 2, NULL);
}
```

8.3 Walkthrough

Global variables

`adc_raw` and `ir_state` are declared outside every function, which places them in global memory and makes them reachable from both tasks. The Sensor Task writes both values, the Output Task reads both values. Previously these were local to the sensor routine, which meant the Output Task had no way to see them. Moving them into global memory is the mechanism the assignment prescribes for this stage, and Section 10.3 examines why it is unsafe.

ADC handle and startup ordering

`adc1_handle` is a global that only becomes valid once `adc_oneshot_new_unit()` has executed inside `setup()`. This is why `setup()` must run before the tasks are created. If the Sensor Task were created first, it could call `adc_oneshot_read()` against an uninitialised handle.

```
app_main() -> setup() -> ADC unit created -> adc1_handle valid
           -> xTaskCreate(sensor_task) -> adc_oneshot_read()
```

PWM configuration

The LEDC timer is configured for 12 bit resolution at 5 kHz. Twelve bits gives $2^{12} = 4096$ duty levels, spanning 0 to 4095, which matches the ESP32 ADC raw range closely enough that `adc_raw` can be handed to `ledc_set_duty()` without any scaling. The 5 kHz frequency means the PWM waveform repeats 5000 times per second; brightness is controlled by varying the duty cycle, not the frequency.

```
ADC raw  0 ... 4095
          |
          v
PWM duty  0 ... 4095      (0% to approximately 100%)
```

Sensor Task

The Sensor Task reads inputs and nothing else. It does not touch the LEDs or the PWM channel. Each pass reads the ADC, checks the returned `esp_err_t`, logs the value at Information level on success or an Error on failure, reads the IR GPIO, logs that state, and then blocks for 100 ms.

Output Task

The Output Task controls outputs and nothing else. It reads the two globals, applies `adc_raw` as the LEDC duty, calls `ledc_update_duty()` to push the new value to the hardware, and drives the digital LED from `ir_state`. The digital write is inverted:

```
gpio_set_level(BLINK_GPIO, !ir_state);

ir_state = 1 -> !ir_state = 0 -> LED OFF   (no object detected)
ir_state = 0 -> !ir_state = 1 -> LED ON    (object detected)
```

The inversion exists because the IR obstacle module used here is active low: it outputs a logic high when nothing is detected and pulls low when an object is present. Reversing the signal makes the LED illuminate on detection, which is the intuitive behaviour.

Task creation

Both tasks are created with a 2048 stack argument and distinct priorities. One ESP-IDF specific detail is worth recording: in vanilla FreeRTOS the stack depth parameter is expressed in words, but ESP-IDF redefines it in

bytes. 2048 here therefore means 2048 bytes, not 8192. This matters when porting stack sizes between a vanilla FreeRTOS project and an ESP-IDF project.

Task	Stack	Priority	Period
Sensor Task	2048 bytes	1 (lower)	100 ms, about 10 Hz
Output Task	2048 bytes	2 (higher)	50 ms, about 20 Hz

Table 7. Task parameters as created in `app_main()`.

8.4 Timing and Why the System Feels Faster

In the earlier version the delays were roughly 1000 ms in the sensor path and 500 ms in the output path. Turning the potentiometer meant waiting up to a second for the new reading and then further time for it to be applied, which felt unresponsive. The delays were reduced to 100 ms and 50 ms, giving roughly ten sensor reads and twenty output updates per second.

The responsiveness improved because the delays were reduced, not because the priorities were changed.

This is a useful thing to have proved by experiment rather than assumed. Priority determines preference among Ready tasks. The delay determines how often a task becomes Ready in the first place. Raising the Output Task's priority without reducing its delay would not have made the system feel any faster.

9. Verification and Serial Monitor Evidence

9.1 What Counts as Proof of Independent Scheduling

The requirement is to show that the two tasks run independently and interleave rather than running one after the other. The ESP-IDF log format supplies this directly, because every line carries both a TAG and a timestamp. Expected output has the following shape:

```
I (1000) SENSOR_TASK: Potentiometer raw value: 1800
I (1002) SENSOR_TASK: IR sensor state: 1
I (1015) OUTPUT_TASK: Output task is running
I (1065) OUTPUT_TASK: Output task is running
I (1100) SENSOR_TASK: Potentiometer raw value: 1950
I (1102) SENSOR_TASK: IR sensor state: 0
I (1115) OUTPUT_TASK: Output task is running
I (1165) OUTPUT_TASK: Output task is running
```

Two properties of that log are the actual evidence. First, the TAGs alternate: the output is not SENSOR_TASK forever followed by OUTPUT_TASK forever, which is what a single sequential execution would produce. Second, the timestamp spacing matches the configured periods, with roughly 100 ms between consecutive SENSOR_TASK groups and roughly 50 ms between consecutive OUTPUT_TASK lines, and roughly two Output lines per Sensor group. Both properties together demonstrate that each task is waking on its own schedule under the control of the scheduler.

9.2 Optional: `vTaskGetRunTimeStats()`

A stronger, quantitative form of the same evidence is available through `vTaskGetRunTimeStats()`, which reports the accumulated run time and the percentage of CPU time consumed by each task. It is not enabled by default and requires the run time statistics options to be turned on in `menuconfig`, under the FreeRTOS component, specifically the trace facility, the statistics formatting functions and run time statistics generation. If it is enabled, the expected result for this project is that both application tasks show a very small percentage and the Idle Task shows the overwhelming majority, which is itself confirmation that neither task is busy waiting.

9.3 Suggested Improvement to the Logging

One weakness in the current implementation is worth noting honestly. The Output Task logs the fixed string "Output task is running", but the assignment asks for the PWM duty cycle being applied to be logged. Changing that line to include the value would satisfy the requirement more directly and would also make the log far more useful when debugging the shared data problem, because the Sensor Task's reported reading and the Output Task's applied duty could then be compared line by line:

```
ESP_LOGI(OUTPUT_TAG, "PWM duty = %d, LED = %d", adc_raw, !ir_state);
```

10. Report Questions

10.1 The Five Task States, and Where Our Tasks Spend Their Time

The five states are Running, Ready, Blocked, Suspended and Deleted. Running means the CPU is executing that task's instructions right now. Ready means the task is able to run and wants CPU time, but something else currently has the CPU. Blocked means the task is waiting for a delay or an event and does not want CPU time until that condition is satisfied. Suspended means the task has been manually taken out of scheduling with `vTaskSuspend()` and will only return when another task calls `vTaskResume()`. Deleted means the task has been removed with `vTaskDelete()` and no longer participates in scheduling at all.

Both of our tasks spend the overwhelming majority of their time Blocked. Neither is ever Suspended or Deleted, since neither of those calls appears in the firmware. The arithmetic makes the point clearly. The Sensor Task blocks for 100 ms per cycle and the Output Task for 50 ms per cycle, while the actual work each performs is far shorter than that. The dominant cost inside each cycle is not the ADC read or the GPIO write, both of which are on the order of microseconds, but the UART transmission of the log lines. At 115200 baud with 8N1 framing, ten bits are transmitted per character, giving roughly 11,520 characters per second, or about 0.087 ms per character. A fifty character log line therefore takes on the order of 4 ms to transmit.

On that basis the Sensor Task, which emits two log lines, is doing perhaps eight or nine milliseconds of work in a 108 ms cycle, and the Output Task, which emits one, perhaps four or five milliseconds in a 54 ms cycle. Both are Blocked for something in the region of ninety percent of their existence. The remainder is split between Running and short spells in Ready, and the Ready periods are brief precisely because the higher priority task keeps taking itself out of contention with its own delay. The typical lifecycle of both tasks is therefore Ready, then Running, then a small amount of work, then `vTaskDelay()`, then Blocked, then Ready again.

10.2 What Priority Decides When Two Tasks Are Both Ready

When two tasks are both Ready, priority decides which one the scheduler hands the CPU to. That is the entire scope of what it controls. It is a preference ordering among the candidates, evaluated at the moment of a scheduling decision.

What priority does not do is equally important. It does not make the task execute faster: the instructions take the same number of cycles either way. It does not make the task run more often: how often a task becomes Ready is set by its delay or by whatever event it is waiting on, not by its priority number. It does not guarantee the task runs first in any absolute sense, because the scheduler only ever chooses among tasks that are actually Ready. A higher priority task that is Blocked is simply not a candidate, and the lower priority task runs instead.

In our project the Output Task at priority 2 outranks the Sensor Task at priority 1. If both are Ready at the same instant, the Output Task is preferred. Because FreeRTOS is preemptive by default, if the Output Task's delay expires while the Sensor Task is mid execution, the scheduler can interrupt the Sensor Task immediately rather than waiting for it to finish. The Sensor Task is not deleted or damaged by that, it is moved back to Ready with its context saved and resumes later from exactly where it stopped.

10.3 How the Output Task Can Read Stale or Inconsistent Data

There are two distinct failure modes here, and it is worth separating them, because they have different causes and different fixes.

Stale data

The Sensor Task updates the globals every 100 ms and the Output Task consumes them every 50 ms. The consumer therefore runs roughly twice as often as the producer, and on approximately every second pass it reads and applies a value that has not changed since the previous pass.

```
t = 0 ms   Sensor writes adc_raw = 2000
t = 50 ms  Output reads 2000, applies duty 2000
t = 100 ms Output reads 2000 again, applies the same duty
t = 100 ms Sensor then updates adc_raw to 2200
t = 150 ms Output finally reads 2200
```

This is benign for an LED, but the general point is that the freshness of the data is not guaranteed by anything in the code. Nothing tells the Output Task whether the value it just read is new or a repeat.

The inconsistent snapshot, which is the real hazard

The Sensor Task does not update both globals in a single operation. It writes `adc_raw` first, then performs a GPIO read, then writes `ir_state`. Between those two writes the scheduler can switch tasks, or on the ESP32 the other core can be executing the Output Task at that very instant.

```
Sensor Task:  adc_raw = NEW
               |
               | <-- scheduler switches, or core 1 runs concurrently
               v
Output Task:  reads adc_raw -> NEW
              reads ir_state -> OLD
               |
               v
Sensor Task:  ir_state = NEW
```

The Output Task has now acted on a pair of values that never existed together as a single sensor reading. It applied a brand new ADC sample alongside a stale IR state. That is a torn snapshot, and it is a real race

condition rather than a theoretical one. In this project the consequence is one wrong LED frame lasting 50 ms, which nobody notices. In a motor control or safety context, acting on a half updated view of the world is exactly the class of bug that causes damage.

Why it happens, precisely

- The two writes are not atomic as a pair. There is no mechanism preventing a task switch or a concurrent read between them, so any reader can observe the intermediate state.
- There is no signal that data is complete. The Output Task has no way to ask whether the Sensor Task is midway through an update, so it reads whatever memory currently holds.
- The variables are not declared volatile. The compiler is entitled to assume that a plain global it can see no writes to inside `output_task` will not change, and may cache it in a register across loop iterations rather than reloading it from memory. That would make the Output Task miss updates entirely at higher optimisation levels.

One nuance is worth stating so that the diagnosis is accurate rather than folkloric. A single 32 bit aligned int on the Xtensa core is loaded and stored by a single instruction, so you will not observe half of an individual integer with some bits old and some bits new. The tearing in this program is at the level of the pair of variables, not within either one. Adding volatile would fix the caching problem but would not fix the pair consistency problem, because volatile constrains the compiler, not the scheduler. The correct fix, which is the subject of the next task, is to make the update atomic with respect to the reader: a queue that carries both readings in one message, a struct protected by a mutex, or a short critical section.

10.4 Why the Dual Core Architecture Matters Here

The ESP32 has two identical Xtensa LX6 cores, CPU0 (also called PRO_CPU) and CPU1 (also called APP_CPU), and ESP-IDF ships an SMP variant of FreeRTOS that schedules across both of them. This changes the shared data question in a way that a single core port would not.

The direct answer to the question is no, our two tasks do not necessarily run on the same core. ESP-IDF provides `xTaskCreatePinnedToCore()` with an `xCoreID` argument, which accepts 0 for CPU0, 1 for CPU1, or `tskNO_AFFINITY` for no affinity. The plain `xTaskCreate()` we used is defined in ESP-IDF as a wrapper that calls `xTaskCreatePinnedToCore()` with `tskNO_AFFINITY`. Our tasks are therefore unpinned, and the scheduler may place either of them on either core. Separately, `app_main()` itself runs on CPU0 by default, controlled by `CONFIG_ESP_MAIN_TASK_AFFINITY`, but the tasks it creates do not inherit that affinity.

Three consequences follow, and they compound the problem described in the previous answer.

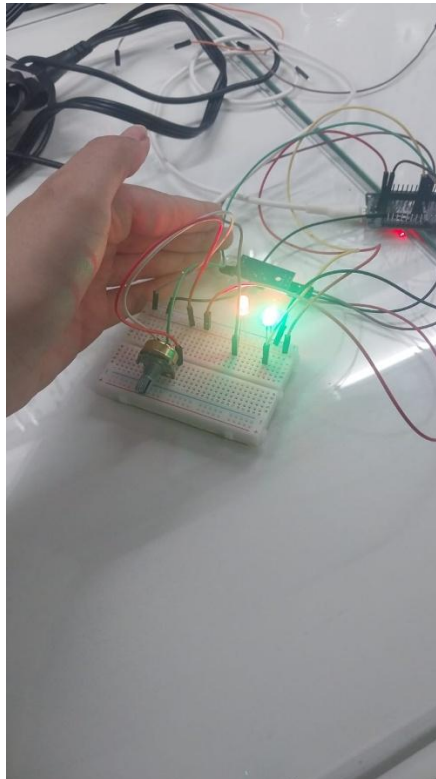
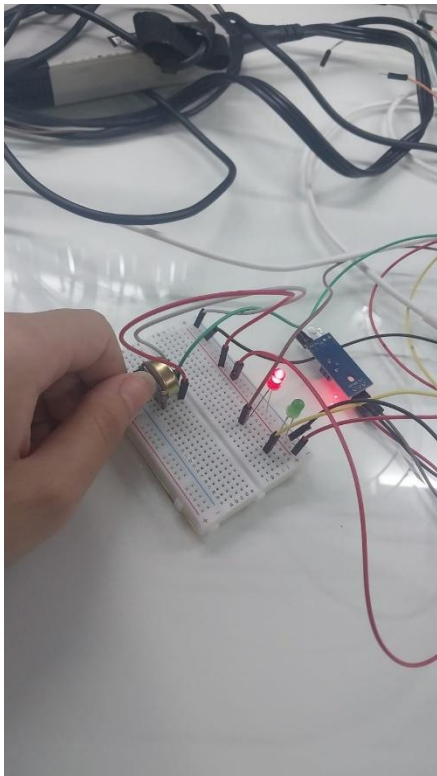
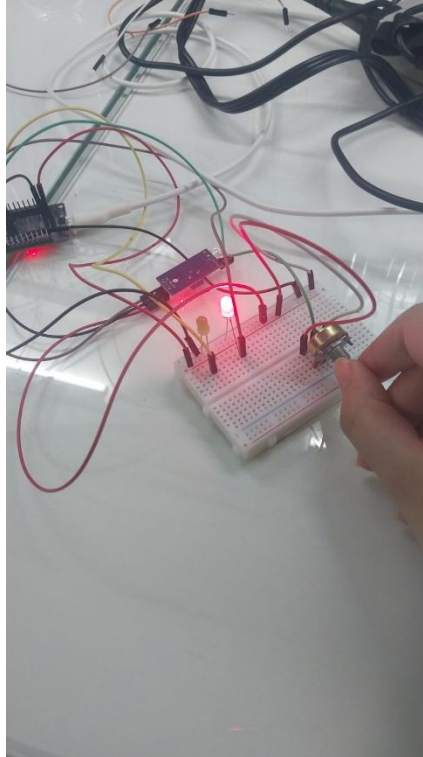
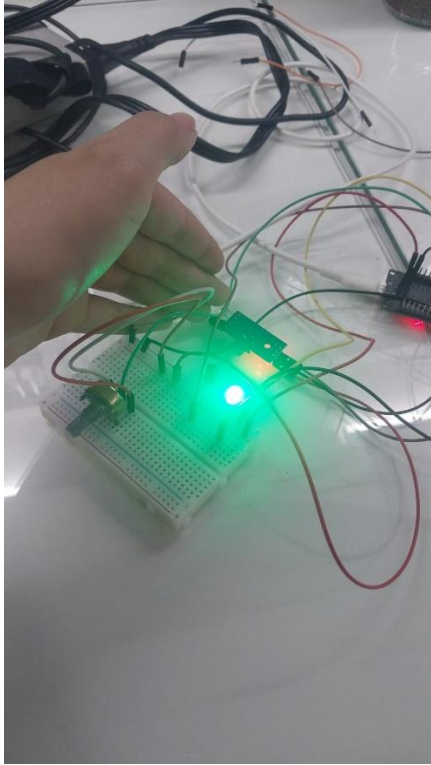
- **The interleaving stops being merely apparent.** On one core, concurrency is an illusion produced by fast switching. With two cores, the Sensor Task and the Output Task can genuinely be executing at the same instant, one on each core. The window between the write to `adc_raw` and the write to `ir_state` is not just a window in which a task switch could occur, it is a window in which another core is actively running.
- **Each core schedules independently.** Each core calls the scheduler separately and selects the highest priority Ready task that it is permitted to run, from a shared ready list. A task already being run by one core cannot be picked up by the other. This means the two highest priority Ready tasks are not always the two that run, and it means priority alone no longer describes the full ordering.

- **Disabling interrupts or suspending the scheduler is not sufficient protection.** Suspending the scheduler in ESP-IDF FreeRTOS only affects the core that made the call. Calling `vTaskSuspendAll()` on CPU0 does nothing to stop CPU1 from scheduling. Any protection for shared data has to be genuinely multi-core safe, which is why the correct tools are mutexes, queues and proper critical sections rather than the single core habits of disabling interrupts. The two cores also do not have synchronised tick interrupts, and CPU0 alone is responsible for keeping time.

The practical conclusion is that the shared global variable pattern used in this task is not merely inelegant on the ESP32, it is genuinely unsafe in the concurrent sense, and the dual core architecture removes the accidental protection that a single core system's cooperative timing would sometimes have provided.

11. Photographic Appendix

Figure 2 in Section 7 is the wiring diagram. The remaining evidence for this submission is photographic and is collected here.



12. Definition of Done

Requirement	Status	Where it is evidenced
Two FreeRTOS tasks created via xTaskCreate() with distinct priorities and stack sizes	Met	Section 8.2, Table 7
Each task self contained with its own infinite loop and vTaskDelay(), no busy waiting	Met	Section 8.2, Section 3.4
Output Task at a higher priority than Sensor Task	Met	Section 4.4, Table 7
Data shared between tasks via a plain global variable, no Queue or Mutex	Met	Section 8.3, Section 10.3
All logging via ESP_LOGI / ESP_LOGW / ESP_LOGE, zero printf calls	Met	Section 6
A distinct TAG per task	Met	Section 6.3
Potentiometer raw value logged	Met	sensor_task, Section 8.2
IR sensor state logged	Met	sensor_task, Section 8.2
PWM duty cycle being applied logged	Partial	See Section 9.3, one line change required
Tasks provably interleave, shown in log timestamps	Met	Section 9.1, Figure 4
Report answers all four questions	Met	Section 10
Schematic reflects the firmware pin usage	Open	See Section 7.3, wiper and IR sensor to be added

Table 8. Definition of Done checklist, with two items honestly flagged as outstanding.

12.1 Carried Forward to the Next Task

- Replace the shared globals with a safe communication mechanism. A queue carrying both readings in a single message solves the torn snapshot problem and the freshness problem at the same time, and is the natural next step.
- Log the PWM duty value rather than a fixed string in the Output Task.
- Complete the schematic: potentiometer wiper to the ADC1 channel, IR sensor to its GPIO, and series resistors on both LEDs.
- Consider whether the tasks should be pinned with xTaskCreatePinnedToCore() once the concurrency behaviour has been studied, either to make the shared data behaviour deterministic or to deliberately exercise the cross core race.

13. References

- FreeRTOS.org, official kernel documentation: task states, scheduling and API reference.
- Espressif, ESP-IDF Programming Guide, FreeRTOS (IDF) and ESP-IDF FreeRTOS (SMP): task creation, core affinity, tskNO_AFFINITY, stack depth units, per core scheduling and scheduler suspension semantics.

- Espressif, ESP-IDF Programming Guide, Application Startup Flow: the main task, CONFIG_ESP_MAIN_TASK_AFFINITY, and the behaviour of app_main() on return.
- Espressif, ESP-IDF Programming Guide, Logging library (esp_log.h): severity levels, tags and log formatting.
- Espressif, ESP-IDF Programming Guide, LEDC (LED PWM Controller) and ADC Oneshot Mode drivers.
- Bhautik Markeye, ESP-IDF logging: how great is esp_logx vs printf, medium.com/@bhautik.markeye/esp-idf-logging-how-great-is-esp-logx-vs-printf-73e24bd6fe78
- Course reference videos and the Day 26 and Day 27 Notion notes, which are the primary source for the structure and worked examples in Sections 2 to 6.